BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

Sudoku Gladiators - Multiplayer Sudoku Game Using Flutter

Supervisor
Lect. dr. Petrescu Manuela-Andreea

Author
Dumitrescu David-Ioan

2025

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ENGLEZĂ

LUCRARE DE LICENŢĂ

Sudoku Gladiators - Un Joc Sudoku Multiplayer folosind Flutter

Conducător științific
Lect. dr. Petrescu Manuela-Andreea

Absolvent
Dumitrescu David-Ioan

2025

ABSTRACT

This thesis presents ”Sudoku Gladiators,” an innovative cross-platform multiplayer Sudoku game built with Flutter and Firebase that transforms the traditionally solitary puzzle into a competitive, real-time gaming experience. **Chapter 1** introduces the project and its core objective: to fill a significant market gap by creating a modern, social, and competitive Sudoku platform. **Chapter 2** establishes the theoretical background by exploring Sudoku's history, its cognitive benefits, its mathematical foundations in combinatorics, and key concepts in multiplayer game design. **Chapter 3** validates the project's direction through comprehensive market research, analyzing search demand, the competitive landscape, and app store data to confirm a strong, unmet user demand for multiplayer Sudoku. **Chapter 4** delves into the theoretical approaches and algorithms that power the game, covering puzzle generation, difficulty assessment, ELO-based matchmaking, and unique solutions for real-time synchronization and conflict resolution within a serverless architecture.

Following the theoretical groundwork, **Chapter 5** details the technologies and frameworks used, highlighting the advantages of Flutter for cross-platform development and the Firebase ecosystem for scalable backend services like authentication and a real-time database. **Chapter 6** outlines the practical implementation, specifying the functional and non-functional requirements, the system architecture, and the detailed game flow for its single-player, ranked, and casual multiplayer modes. **Chapter 7** focuses on the UI/UX design, presenting a design philosophy that balances competitive information with puzzle clarity and providing a screen-by-screen analysis of the user journey, from lobbies to the final game interface. Finally, **Chapter 8** concludes the thesis by summarizing the project's achievements, discussing its limitations, and proposing future work, reinforcing the finding that serverless architectures can effectively power complex, real-time competitive puzzle games.

Contents

Chapter 1

Introduction

Sudoku is a widely recognized and beloved logic-based puzzle game that has captivated an audience of millions worldwide. For those who do not know, in a classic Sudoku [Stu23] puzzle, the objective is to fill a 9x9 grid with digits such that each row, column and each of the nice 3x3 subgrids are filled with the digits from 1 to 9. Its general appeal lies in the simplicity of its rules and in the challenge that it poses, making it accessible to casual novices and serious puzzle enthusiasts. Historically, Sudoku puzzles have been single-player, offering a solitary experience focused on logic, indivdual skill and experince. However, seeing the growth in popularity of competitve online gaming, we have the exciting opportunity to create a new social and interactive experience.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 1.1: A classic Sudoku puzzle [Wik25]

My motivation for undertaking this project comes from a personal passion for solving Sudoku puzzles and for competitive gaming. I remember being a young kid and discovering that my grandmother's Nokia phone came with Sudoku and

spending countless hours and days trying to figure out how it worked and strategies that I could use to improve my times. But, the only thing that didn't sit right with me was that I had to do it alone, so the opportunity of creating a multiplayer Sudoku experience is of particular interest to me.

This Bachelor's thesis explores the development of "Sudoku Gladiators", an innovative multiplayer Sudoku game that leverages the cross-platform capabilities of Flutter. The project aims to enhance traditional Sudoku gameplay by integrating competitive, real-time multiplayer modes that hope to add a new layer of strategy and excitement to the classical puzzle.

The primary objectives of this thesis are:

- Design and implement multiple multiplayer game modes that will be appealing to both casual and competitve players
- Develop robust matchmaking systems and engaging gameplay mechanics using real-time synchronization strategies

The paper is structured to take the reader through the theoretical foundations of Sudoku and multiplayer gaming, examine relevant algorithmic approaches to puzzle solving, discuss technological frameworks utilized in the project, detail the practical implementation, and evaluate the game's performance and user experience. Finally, the conclusion synthesizes the project's outcomes, reflecting on achievements and proposing directions for future development.

1.1 Declaration of Generative AI and AIassisted technologies in the writing process

During the preparation of this work, the author used Claude and ChatGPT in order to assist with literature research, grammar checking, and organizing content into appropriate chapter structures. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Chapter 2

Theoretical Background

2.1 History of Sudoku

Sudoku, originally named ”Number Place”, was first developed by American architect Howard Garns and published in 1979 by Dell Magazines [Smi05]. The puzzle was a numeric variation of Latin squares, a concept linked to Leonhard Euler. However, some claim the korean mathematician Choi Seok-jeong had fist published an example of the Latin square. Although Sudoku found moderate success in the United States under its original name, its international breakthrough came only 5 years later when it was introduced to the Japanese market by Nikoli Co., Ltd. under the name ”Sudoku” meaning ”single number”.

The simplicity and elegance of Sudoku’s rules, combined with the increasing availability of personal computers and mobile devices, fueled its explosive popularity in the early 2000s. Its widespread inclusion in newspapers and the release of digital Sudoku games on various platforms helped solidify its status as one of the most iconic logic puzzles of the 21st century.

2.2 Cognitive benefits and Educational Value

Beyond entertainment, Sudoku has been shown to offer significant cognitive benefits. Studies have indicated that solving Sudoku puzzles can improve memory, concentration, and logical reasoning abilities. According to research published in the journal ”Activities, Adaptation & Aging”, engaging regularly in logic-based games like Sudoku may help maintain cognitive functioning in older adults and delay the onset of cognitive decline [and11]. For students, Sudoku can be a useful tool to enhance problem-solving skills and support learning in mathematics by reinforcing pattern recognition and analytical thinking.

Additionally, educators sometimes incorporate Sudoku into classroom activities

to stimulate interest in logical reasoning and to promote a hands-on approach to abstract problem-solving. The game's adaptability to different difficulty levels makes it suitable for various age groups and learning objectives [Jin22].

2.3 Mathematical Foundations of Sudoku

Sudoku is fundamentally based on mathematical principles, especially those concerning combinatorics and constraint satisfaction. The puzzle can be considered a special case of the Latin square, a grid where each number appears only once per row and column, with the additional constraint that each 3x3 subgrid must also contain unique digits.

Mathematically, solving a Sudoku puzzle is a form of contraint satisfaction, where the goal is to assign values to a set of cells while satisfying the previously stated constraints. This has made the sudku puzzle quite useful in fields such as artificial intelligence and algorithm design.

2.4 Multiplayer Game Overview

Multiplayer games have evolved significantly alongside advancements in networking technology. These games enable users to interact and compete in real-time or asynchronously, forming one of the most engaging sectors of the gaming industry. According to Unity's player report, more than half of the global population (52%) play games, and of those gamers, 77% play multiplayer games[Uni22].

Real-time multiplayer games require fast and synchronized communication between clients and servers to maintain a consistent game state. This is often achieved using WebSocket technology or custom protocols built on TCP/UDP. Turn-based multiplayer games, on the other hand, typically tolerate higher latency and are less resource-intensive, allowing players to take their turns at their convenience.

Matchmaking systems play a crucial role in multiplayer games, particularly in competitive environments. These systems often rely on algorithms like the Elo rating system [Pow23], which dynamically adjusts player ratings based on wins, losses, and the skill level of opponents. This ensures fair and balanced pairings, improving the overall player experience.

In this context, integrating multiplayer elements into a Sudoku game represents a unique challenge. It requires not only maintaining puzzle integrity across sessions but also managing competitive fairness, tracking player statistics, and ensuring a seamless user experience across devices.

Chapter 3

Market Research and Analytics

3.1 Overview

Market research reveals significant opportunities in multiplayer Sudoku gaming through analysis of search demand, competitive landscape, and app store performance. This chapter examines market gaps and validates user demand for innovative multiplayer puzzle experiences.

3.2 Search Demand Analysis

Google Trends analysis shows stable interest in "sudoku" searches (80-95 points) versus high volatility in "sudoku multiplayer" searches (peaks at 100, drops to 25-75). This pattern indicates users actively seeking multiplayer solutions but not finding satisfactory options, validating unmet market demand.

3.3 Academic Research Foundation

3.3.1 SudoDuel Research

Academic research on SudoDuel[Tud24] validated the technical feasibility of real-time multiplayer Sudoku with strategic power-up systems. The study demonstrated successful implementation of nine strategic abilities across three cost tiers, providing a proven framework for competitive Sudoku mechanics.

3.3.2 Clash of Sudokers Research

Research on Clash of Sudokers[Sok24] explored real-time competitive multiplayer approaches, focusing on same-board competition mechanics and machine learning-

based ranking systems. The study developed innovative cell-locking mechanics where players compete directly on shared puzzles, with correct moves reflected on opponents' grids. This research contributed to understanding direct competitive gameplay in puzzle environments and validated advanced player rating algorithms using multiple performance factors.

3.3.3 Research Validation

Both academic projects confirm that multiplayer Sudoku can maintain puzzle integrity while adding strategic depth, addressing concerns about gameplay balance and technical feasibility.

3.4 Competitive Landscape

3.4.1 Current Market Players

UsDoku operates as a web-based platform offering real-time multiplayer Sudoku where players compete to complete puzzles fastest, with blocks changing color to indicate who solved them first. Operating on a donation-based model, it demonstrates the monetization challenges facing current multiplayer implementations.

Live Sudoku implements sophisticated competitive mechanics where players can see opponent boards with color-coded feedback: correct digits, wrong digits, and pencil marks are visually distinguished without revealing specific numbers. This approach maintains puzzle integrity while providing competitive awareness, representing one of the more advanced current implementations.

Sudokill, developed at New York University, takes a unique turn-based approach where players attempt to force opponents into invalid moves. Players must place numbers in the same row or column as opponent's previous moves, creating strategic positioning gameplay. While innovative, its academic origins and turn-based nature limit mainstream appeal.

Sudoku Battles positions itself as competitive puzzle-solving races between friends and includes a progression-based powerup system where players earn abilities through gameplay advancement. However, players can only hold one powerup at a time and do not compete for powerups during matches, limiting strategic depth compared to time-based competitive powerup systems. While this represents some innovation beyond basic racing, the restricted powerup mechanics reduce tactical complexity and real-time strategic decision-making.

3.4.2 Market Gaps

Analysis reveals that existing players suffer from critical limitations that create substantial market opportunities:

1. **Limited Strategic Depth**: Most platforms focus on basic speed competition without innovative mechanics for sustained engagement. Current solutions fail to address the fundamental problem that traditional Sudoku solving becomes repetitive over time. This creates an opportunity for implementations featuring competitive powerup systems, time-based strategic elements, and adaptive difficulty mechanics that maintain long-term player interest.

2. **Monetization Challenges**: Platforms struggle with sustainable revenue models, relying primarily on donations or academic funding rather than proven freemium approaches. This demonstrates market immaturity and creates opportunities for applications implementing successful mobile gaming revenue strategies including premium features, cosmetic customization, and skill-based progression systems.

3. **Technical Limitations**: Many existing solutions lack cross-platform compatibility and advanced networking infrastructure necessary for competitive gaming. Poor technical implementation results in connectivity issues and limited accessibility across devices. This creates opportunities for robust implementations featuring real-time synchronization and seamless cross-platform play.

4. **Niche Appeal and User Experience**: Current solutions serve primarily tech-focused audiences rather than mainstream gaming markets. Existing platforms often feature outdated user interfaces and poor user experience design that fails to attract casual players. This creates significant opportunities for consumer-focused applications featuring intuitive interfaces, streamlined onboarding, and design principles that appeal to broader demographics.

5. **Social and Community Features**: Most existing platforms lack robust social features and community building tools common in successful mobile games. The absence of friend systems, leaderboards, and social sharing limits viral growth and long-term engagement. This presents opportunities for implementations emphasizing community building and social competition.

3.4.3 Mobile App Store Analysis

Analysis of the mobile app store validates the significant market opportunity for multiplayer Sudoku. This opportunity exists within the context of the broader puzzle game category, a powerhouse in the mobile gaming industry. In 2024, the puzzle

genre saw a significant increase in both downloads and revenue, reportedly reaching 7.53 billion downloads and $6.05 billion in revenue[Fox25]. This immense scale underscores the financial viability of the market this project aims to enter. Within the Sudoku sub-genre itself, leading applications like Sudoku.com have achieved over 50 million downloads and maintain exceptional user satisfaction ratings of 4.8-4.9 stars. These top applications successfully implement freemium monetization models -generating millions in revenue through strategies such as premium subscriptions, ad removal, and hint purchases - which confirms the commercial viability of the niche.

Despite this proven demand and successful monetization in the single-player space, no leading application has established itself as a real-time multiplayer leader. This represents a clear and substantial market gap. The absence of a dominant multiplayer platform in such a large market creates an exceptional opportunity for a first-mover advantage. This gap is particularly significant given the broader mobile gaming trend of integrating social and competitive features to drive long-term user engagement and build community.

Therefore, the app store data reveals ideal conditions for competitive positioning. The potential to convert even a small fraction of the existing, highly engaged single-player audience through multiplayer innovation presents a clear path to acquiring millions of users and generating substantial revenue. This untapped potential within a proven market strongly validates the strategic opportunity for this project.trategic opportunity for this project.

3.5 Conclusion

Market research confirms exceptional opportunities in multiplayer Sudoku gaming. The analysis reveals strong user demand through search patterns and app store performance, while academic research validates technical feasibility. Current competitive solutions suffer from significant limitations in strategic depth, monetization, technical implementation, user experience, and social features.

Most importantly, despite a substantial existing market with proven commercial success, no leading applications offer real-time multiplayer experiences. This represents a clear market gap in a proven, profitable gaming category with millions of engaged users.

The convergence of demonstrated demand, technical validation, competitive weaknesses, and absence of multiplayer market leaders creates optimal conditions for innovative multiplayer Sudoku implementations.

Chapter 4

Theoretical approaches and algorithm analysis

This chapter examines the fundamental algorithms and theoretical approaches that underpin Sudoku puzzle solving, validation, and grading, before addressing the unique challenges of multiplayer implementation using a Firebase-centric architecture. The analysis begins with a comprehensive overview of established Sudoku solving methodologies from the literature, their computational complexities and practical applications, followed by an examination of puzzle validation and difficulty assessment techniques. The chapter then transitions to multiplayer-specific algorithms including real-time synchronization, matchmaking, and anti-cheat mechanisms implemented entirely within the Firebase ecosystem.

4.1 Classical Sudoku Solving Algorithms

The computational study of Sudoku solving has evolved significantly since the puzzle's popularization in the early 2000s. Various algorithmic approaches have been developed, each with distinct characteristics regarding completeness, efficiency, and implementation complexity. This section provides a theoretical foundation by examining established solving methodologies before detailing their application in the Sudoku Gladiators system.

4.1.1 Overview of Solving Strategies

Constraint Propagation Methods

Constraint propagation approaches leverage the inherent logical structure of Sudoku puzzles. Norvig [Nor07] demonstrated that systematic constraint propagation

could solve approximately 95% of newspaper Sudoku puzzles without backtracking. The fundamental operations include:

- **Elimination**: Removing impossible values from cells based on row, column, and box constraints
- **Only Choice**: Identifying cells where only one value remains possible
- **Naked Singles**: Finding cells with exactly one candidate value
- **Hidden Singles**: Locating values that can only appear in one position within a unit

Backtracking Algorithms

Backtracking represents the classical complete search approach for constraint satisfaction problems. Crook [Cro09] provided a comprehensive analysis showing that pure backtracking has worst-case exponential complexity but performs well on typical Sudoku instances due to the problem's structure.

Hybrid Human-Like Strategies

Several researchers have developed algorithms that mirror human solving techniques. Simonis [Sim05] catalogued over 40 distinct human solving patterns, ranging from basic eliminations to advanced techniques like "X-wings" and "swordfish" patterns.

Stochastic and Metaheuristic Approaches

Alternative approaches include genetic algorithms [MK07], simulated annealing [Lew07], and other metaheuristic methods. While these approaches are not guaranteed to find solutions, they can be useful for puzzle generation and analysis.

4.1.2 Comparative Analysis of Solving Approaches

To establish the theoretical foundation for our Sudoku engine implementation, a comprehensive comparison of established solving methodologies is essential. Table 4.1 will present a detailed analysis of various algorithmic approaches, examining their completeness guarantees, computational complexity, and practical performance characteristics to justify our implementation choices.

Table 4.1: Comparative Analysis of Sudoku Solving Algorithms

Algorithm Class	**Completeness**	**Time Complexity**	**Space Complexity**	**Typical Success Rate**	**Refs**
Pure Constraint Propagation	Incomplete	$O(n^2)$ per iter.	$O(n^2)$	∼95% (easy)	[Nor07]
Basic Backtracking	Complete	$O(9^k)$ worst	$O(k)$	100%	[Cro09]
Backtracking + Propagation	Complete	$O(9^k)$ worst	$O(n^2)$	100% (faster)	[Nor07]
Human-like Strategies	Incomplete	$O(n^3)$ per tech.	$O(n^2)$	Variable	[Sim05]
AC-3 Constraint	Complete	$O(n^3)$	$O(n^2)$	100%	[Mac77]
Dancing Links	Complete	$O(9^k)$ worst	$O(n^3)$	100% (opt.)	[Knu00]
Genetic Algorithm	Incomplete	Probabilistic: $O(g * p * (C_f + C_o))$	$O(p * n^2)$	Variable	

where $n = 9$ (grid size), $k =$ empty cells, $g =$ number of generations, $p =$ population size, $C_f =$ cost of fitness function per individual, $C_o =$ cost of applying genetic operators on an individual

4.1.3 Theoretical Foundation for Implementation Choice

The Sudoku Gladiators implementation adopts a hybrid approach combining randomized backtracking with constraint validation for several theoretical and practical reasons:

Randomized Backtracking Selection

The choice of randomized backtracking over pure constraint propagation stems from the need for puzzle generation rather than just solving. As demonstrated by McGuire et al. [MTC14], generating minimal Sudoku puzzles requires complete search capabilities to ensure uniqueness properties.

Pseudocode for Randomized Generation

Algorithm 1 GenerateRandomizedSudoku

Require: Empty 9×9 grid G

Ensure: Complete valid Sudoku grid

for each cell (i, j) in G (row-major order) **do**
  if $G[i, j]$ is empty **then**
    $candidates \leftarrow$ SHUFFLE($\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$)
    for each value v in $candidates$ **do**
      if IsValid(G, i, j, v) **then**
        $G[i, j] \leftarrow v$
        if GenerateRandomizedSudoku(G) **then return** true
        end if
        $G[i, j] \leftarrow 0$ ▷ backtrack
      end if
    end for
  end if
end forreturn false
function IsValid($G, row, col, value$) **return not** ($value$ in $G[row, *]$ **or** $value$ in $G[*, col]$ **or** $value$ in GetBox(G, row, col))
end function

This approach ensures $O(1)$ validation per move while maintaining the completeness property required for puzzle generation.

4.2 Puzzle Difficulty Assessment

Difficulty assessment in Sudoku puzzles is a complex problem that intersects computational complexity theory with human cognitive science. The challenge lies in creating objective measures that correlate with subjective human difficulty perception.

4.2.1 Theoretical Approaches to Difficulty Measurement

Clue-Based Metrics

The most basic approach measures difficulty by the number of given clues. Felgenhauer and Jarvis [FJ06] established that valid Sudoku puzzles contain between 17 and 80 clues, but this metric poorly correlates with solving difficulty.

Complexity-Based Metrics

More sophisticated approaches analyze the logical complexity required to solve puzzles. The technique hierarchy from Simonis [Sim05] provides a foundation:

1. **Level 1**: Basic elimination and naked singles
2. **Level 2**: Hidden singles and intersection removal
3. **Level 3**: Subset techniques (pairs, triples)
4. **Level 4**: Advanced patterns (X-wing, swordfish)
5. **Level 5**: Complex chains and forcing patterns

Search Space Analysis

While early approaches to puzzle difficulty assessment often focused on properties like the number of initial clues or the size of the algorithmic search space, Pelánek[Pel14] provides strong empirical evidence from thousands of human-solved Sudoku puzzles that these measures are poor predictors of cognitive load. His work demonstrates a paradigm shift, showing that difficulty is most accurately captured by cognitive models that account for two key factors: the intrinsic complexity of the logical deductions required and the dependency structure of the solution path, with such models achieving a correlation with human solving times as high as 0.95.

4.2.2 Implementation Strategy

The Sudoku Gladiators system employs a cell removal strategy with validation to control difficulty:

Algorithm 2 GenerateDifficultyControlledPuzzle

Require: Complete grid G, target difficulty D
Ensure: Puzzle P with controlled difficulty

$P \leftarrow \text{COPY}(G)$
$cellsToRemove \leftarrow \text{GetTargetRemovalCount}(D)$
$positions \leftarrow \text{SHUFFLE}(\text{AllCellPositions}())$
$removed \leftarrow 0$
for each position (i, j) in $positions$ **do**
    if $removed \geq cellsToRemove$ **then break**
    end if
    $backup \leftarrow P[i, j]$
    $P[i, j] \leftarrow 0$
    if HASUNIQUESOLUTION(P) **then**
        $removed \leftarrow removed + 1$
    else
        $P[i, j] \leftarrow backup$ ▷ restore if ambiguous
    end if
end forreturn P

The uniqueness validation employs a bounded search that terminates after finding two solutions, ensuring $O(9^k)$ worst-case complexity but typically performing much better due to early termination.

4.3 Multiplayer Synchronization Algorithms

Multiplayer Sudoku introduces significant challenges in distributed systems theory, particularly regarding consistency, availability, and partition tolerance as defined by the CAP theorem [Bre00].

4.3.1 Consistency Models for Real-Time Gaming

Technical Strong Consistency vs. Eventual Consistency

Traditional game servers often implement strong consistency to prevent cheating and ensure fair play. However, this approach introduces latency that can degrade user experience. Firebase Firestore provides strong consistency for single-document operations while maintaining eventual consistency across the distributed system.

Conflict Resolution in Concurrent Moves

The theoretical challenge arises when multiple players attempt moves simultaneously. The system must determine a canonical ordering while maintaining fairness. The implemented solution uses Firestore's server timestamps to establish canonical ordering:

Algorithm 3 AtomicFirstPlaceClaim

Require: $lobbyId$, $playerId$, $completionTime$

Ensure: Boolean indicating first place claim success

1: **begin transaction**
2: $firstPlaceDoc \leftarrow$ READ($lobbies/lobbyId/gameResults/firstPlace$)
3: **if** $firstPlaceDoc.exists$ **then return** false ▷ already claimed
4: **else**
5: WRITE($firstPlaceDoc$, $\{playerId, serverTimestamp\}$)
6: **commit transaction return** true
7: **end if**

This approach guarantees that exactly one player can claim first place, regardless of network latency or client-side timing variations.

4.3.2 Theoretical Properties of Firebase-Based Synchronization

Linearizability Properties

Firestore transactions provide linearizability for single-document operations, which is sufficient for the critical operations in Sudoku Gladiators:

- Move validation and logging
- First place determination
- Powerup claiming

Network Partition Handling

The Firebase architecture handles network partitions through its multi-region replication system. Clients automatically reconnect and resynchronize when connectivity is restored, maintaining the illusion of continuous availability.

4.4 Matchmaking Algorithm Design

Multiplayer matchmaking represents a complex optimization problem balancing multiple objectives: skill balance, waiting time minimization, and player satisfaction.

4.4.1 Elo Rating System Adaptation

The traditional Elo rating system [Elo78] requires adaptation for Sudoku competition. The fundamental equation:

$$R'_A = R_A + K(S_A - E_A) \tag{4.1}$$

Where:

- R'_A is the new rating for player A
- R_A is the current rating for player A
- K is the K-factor (learning rate)
- S_A is the actual score (1 for win, 0 for loss)
- E_A is the expected score: $\frac{1}{1+10^{(R_B-R_A)/400}}$

K-Factor Selection

The choice of K-factor significantly impacts rating stability and responsiveness. Research by Glickman [Gli99] suggests $K = 32$ for established systems, which Sudoku Gladiators adopts.

4.4.2 Queue-Based Matchmaking Algorithm

The matchmaking system implements an expanding search radius approach:

Algorithm 4 ExpandingRadiusMatchmaking

Require: $playerId$, $currentTime$

Ensure: $opponentId$ or NULL

1: $playerRating \leftarrow$ GetPlayerRating($playerId$)
2: $queueTime \leftarrow currentTime -$ GetJoinTime($playerId$)
3: $searchRadius \leftarrow \min($INITIAL_RADIUS$+\lfloor queueTime /$ EXPANSION_INTERVAL$\rfloor \times$ EXPANSION_RATE, MAX_RADIUS$)$
4: $candidates \leftarrow []$
5: **for** each $opponent$ in $rankedQueue$ **do**
6: **if** $|opponent.rating - playerRating| \leq searchRadius$ **then**
7: $candidates$.ADD($opponent$)
8: **end if**
9: **end for**
10: **if** $candidates$.isEmpty() **then return** NULL
11: **elsereturn** SELECTCLOSESTRATINGMATCH($candidates$, $playerRating$)
12: **end if**

This algorithm provides theoretical guarantees:

- **Bounded waiting time**: Maximum queue time is bounded by MAX_RADIUS expansion
- **Skill balance**: Initial matches prioritize rating proximity
- **Fairness**: All players experience identical expansion curves

4.4.3 Complexity Analysis of Matchmaking Operations

The matchmaking system's performance characteristics are critical for maintaining reasonable wait times and fair player pairing. The detailed complexity analysis of individual matchmaking operations will be discussed in Table 4.2, which breaks down the computational and network overhead for each core matchmaking function.

Table 4.2: Matchmaking Operations Complexity

Operation	Time Complexity	Space Complexity	Network Operations
Join Queue	$O(1)$	$O(1)$	1 write
Find Match	$O(n)$	$O(n)$	1 read
Rating Update	$O(1)$	$O(1)$	1 write
Queue Cleanup	$O(n)$	$O(1)$	n writes (batched)

where $n =$ number of players in queue

4.5 Cooperative Game Mode Design

Beyond competitive multiplayer, the system architecture supports a cooperative mode where two players collaborate to solve a single puzzle. This mode introduces unique theoretical challenges in distributed coordination and shared state management.

4.5.1 Shared State Synchronization

Conflict Resolution in Collaborative Editing

Cooperative Sudoku presents similar challenges to collaborative document editing systems. When two players attempt to modify the same cell simultaneously, the system must resolve conflicts while maintaining puzzle validity:

Algorithm 5 CooperativeConflictResolution

Require: cell(i, j), $player1Value$, $player2Value$, $timestamps$
Ensure: Resolved cell value
if $timestamp1 < timestamp2$ **then**
    $resolvedValue \leftarrow player2Value$
    $conflictWinner \leftarrow player2$
else
    $resolvedValue \leftarrow player1Value$
    $conflictWinner \leftarrow player1$
end if
if IsValidSudokuMove($puzzle$, i, j, $resolvedValue$) **then**
    Apply($resolvedValue$)
    NotifyPlayer($conflictLoser$, "Move overridden")
else
    Reject($resolvedValue$)
    NotifyPlayer($conflictWinner$, "Invalid move")
end if

Operational Transformation for Sudoku

The cooperative mode can benefit from operational transformation theory [EG89], adapting document editing algorithms for Sudoku constraints. Each move operation must be transformable against concurrent operations while preserving puzzle validity.

4.5.2 Coordination Mechanisms

Turn-Based vs. Simultaneous Editing

Two primary coordination models exist:

1. **Strict Turn-Based**: Only one player can make moves at a time, eliminating conflicts but reducing engagement
2. **Optimistic Concurrent**: Both players can attempt moves simultaneously with conflict resolution

The system implements optimistic concurrency with last-write-wins semantics, balanced by move validation to prevent invalid states.

Shared Resource Management

Cooperative mode introduces shared resource pooling between players working on the same puzzle:

- **Combined hint pool**: Both players share a total hint allocation (e.g., 6 hints total instead of 3 per player)
- **Unified mistake tracking**: Shared mistake counter with combined limit
- **Collaborative progress**: Both players contribute to the same completion goal
- **Synchronized game state**: All moves and progress updates apply to the shared puzzle instance

The resource sharing mechanism ensures fair access while maintaining the collaborative nature of the gameplay. Either player can use available hints, but the total pool is finite and shared between both participants.

4.6 Powerup System Implementation

The powerup system introduces additional complexity by overlaying special abilities onto the base Sudoku mechanics. This creates a hybrid system combining deterministic puzzle logic with strategic ability usage.

4.6.1 Fairness in Powerup Distribution

Pre-Generated vs. Dynamic Spawning

Dynamic powerup spawning can introduce unfairness due to timing advantages. The implemented pre-generated approach ensures deterministic fairness:

Algorithm 6 FairPowerupScheduling

Require: $lobbyId$, $gameMode$, $puzzleLayout$

Ensure: Scheduled powerup spawn list

if $gameMode \neq$ POWERUP **then**
    return empty
end if
$emptyCells \leftarrow$ ExtractEmptyCells($puzzleLayout$)
$locations \leftarrow$ SHUFFLE($emptyCells$).TAKE(MAX_POWERUPS)
$times \leftarrow$ GenerateUniformTimes(MIN_INT, MAX_INT, MAX_POWERUPS)
$schedule \leftarrow []$
for $i \leftarrow 0$ **to** MAX_POWERUPS $- 1$ **do**
    $type \leftarrow$ RandomPowerupType()
    $schedule$.ADD($\{loc : locations[i], time : times[i], type : type\}$)
end for
STORE(lobbies/lobbyId/powerupSchedule, schedule)
return $schedule$

Claiming Race Conditions

Multiple players may attempt to claim the same powerup simultaneously. The system resolves this through atomic claiming:

Algorithm 7 AtomicPowerupClaim

Require: $lobbyId$, $powerupId$, $playerId$

Ensure: Boolean indicating claim success

1: **begin transaction**
2: $powerupDoc \leftarrow$ READ($lobbies/lobbyId/powerups/powerupId$)
3: **if** $powerupDoc.claimedBy \neq$ NULL **then return** false ▷ already claimed
4: **else**
5: UPDATE($powerupDoc$, $\{claimedBy : playerId, claimedAt : serverTimestamp\}$)
6: CREATE(lobbies/lobbyId/playerPowerups/$\{playerId\_timestamp\}$, powerupData)
7: **commit transaction return** true
8: **end if**

4.6.2 Powerup Effect Implementation Strategy

Different powerup types require different implementation approaches based on their temporal and spatial characteristics:

- **Immediate Effects** (Reveal Cells, Extra Hints): Applied instantly to local game state
- **Timed Effects** (Freeze, Solution Show): Stored with expiration timestamps
- **Persistent Effects** (Shield): Maintained until consumed or expired
- **Area Effects** (Bomb): Applied to spatial regions with conflict resolution

4.7 Security Analysis

The serverless architecture requires careful consideration of security. The model assumes that clients cannot be trusted, so all critical game logic is validated on the server side by checking move validity, detecting impossibly fast actions, and verifying progress consistency.

This server-side approach is designed to directly mitigate known security risks. The primary concerns include **speed hacking**, where a player uses automation to accelerate input; **state manipulation**, which involves attempts to modify game data directly; **timing attacks** that exploit network latency; and **player collusion**.

To further secure gameplay, all actions are marked with a server-generated timestamp, preventing players from manipulating the timing of their moves.

4.7.1 Theoretical Security Properties

Information-Theoretic Security

The puzzle solution is never transmitted to clients, providing information-theoretic security against solution extraction attacks. Only progress updates and validation results are communicated.

Cryptographic Timestamps

Server-generated timestamps provide cryptographic ordering guarantees, preventing clients from manipulating temporal aspects of gameplay.

4.8 Performance and Scalability Analysis

Asymptotic Complexity Analysis

The theoretical performance characteristics of key system components have been analyzed to ensure scalability and efficiency across all game modes. Table 4.3 presents a comprehensive breakdown of time, space, and network complexity for each critical system component, providing insights into the system's theoretical performance bounds and scalability properties.

Table 4.3: System Components Complexity Analysis

Component	**Time**	**Space**	**Network**
Puzzle Generation	$O(9^k)$ exp. $O(k^3)$	$O(n^2)$	N/A
Move Validation	$O(n)$	$O(1)$	$O(1)$
Real-time Sync	$O(1)$ per update	$O(p \times n^2)$	$O(p)$
Coop. Conflict Res.	$O(1)$	$O(1)$	$O(1)$
Matchmaking	$O(q)$	$O(q)$	$O(1)$
Powerup System	$O(1)$ per op.	$O(p)$	$O(1)$

where k =empty cells, n =grid size (9), p =players, q =queue size

Scalability Theoretical Bounds

The system's Firebase-based architecture is designed for linear scaling across most operations. The theoretical bounds are defined by the underlying Firestore database, which supports up to 10,000 writes per second, 1,000,000 concurrent connections, a maximum document size of 1 MiB, and a subcollection depth of 100

levels. Despite these high capacities, the primary bottleneck at scale is the matchmaking algorithm, which has a search complexity of $O(n)$.

4.9 Conclusion

The theoretical analysis reveals that Firebase-centric multiplayer game architectures can achieve competitive performance characteristics while significantly simplifying system complexity. Key theoretical contributions include:

1. **Atomic Consistency**: Firestore transactions provide sufficient guarantees for fair competitive and cooperative gaming
2. **Scalable Synchronization**: Real-time listeners eliminate polling overhead while maintaining consistency in both competitive and collaborative modes
3. **Conflict Resolution**: Operational transformation principles adapted for Sudoku enable smooth cooperative gameplay
4. **Security Properties**: Declarative security rules provide equivalent protection to traditional server validation
5. **Algorithmic Efficiency**: Hybrid solving approaches optimize for both generation and validation requirements

The next chapter will examine the specific technological implementation of these theoretical concepts within the Flutter and Firebase ecosystem.

Chapter 5

Technologies and Frameworks

This chapter presents the primary technologies and frameworks utilized in developing the "Sudoku Gladiators" multiplayer game. The focus is on building a cross-platform, responsive, and real-time gaming experience using a modern serverless architecture. The technology stack was chosen to streamline development, ensure scalability, and provide a seamless experience for users across all platforms while leveraging cloud-native solutions.

5.1 Flutter Framework

Flutter is an open-source UI framework developed by Google for building natively compiled applications across mobile (iOS, Android), web, and desktop platforms from a single codebase [Goo25c]. It leverages the Dart programming language and provides a fast development cycle via features like hot reload, expressive UI, and a comprehensive widget catalog.

5.1.1 Advantages for Cross-Platform Development

Flutter's single codebase approach reduces development time significantly and ensures feature parity across platforms, as both UI and business logic are shared between all target platforms. The framework achieves high performance by using the Skia graphics engine for native speed, smooth animations, and consistent rendering across different devices and operating systems.

The rich widget ecosystem provides pre-built and customizable components that support complex layouts, accessibility features, and adaptive UI for different screen sizes. Flutter maintains an active community that provides frequent updates, strong community support, and extensive documentation that help accelerate development processes. The hot reload feature enables developers to view code changes

instantly without losing application state, significantly speeding up the development and debugging process.

5.1.2 State Management with Provider

The application uses the Provider pattern for state management [Rem25], offering several key advantages for maintaining application state. Reactive updates automatically rebuild UI components when underlying data changes, ensuring the interface always reflects the current application state. The pattern promotes separation of concerns by keeping business logic separate from UI components, making the codebase more maintainable and testable.

State management logic can be easily unit tested in isolation from UI components, improving code quality and reliability. The Provider pattern also delivers performance benefits by ensuring only affected widgets rebuild when state changes occur, minimizing unnecessary UI updates. Key providers implemented include SudokuProvider for managing game board state, moves, and validation, LobbyProvider for handling multiplayer lobby creation, joining, and management, PowerupProvider for managing powerup spawning, claiming, and effects, and ThemeProvider for controlling application theming and appearance.

5.2 Dart Programming Language

Dart is a modern, object-oriented language optimized for UI development, concurrency, and portability [Goo25a]. It supports both ahead-of-time (AOT) and just-in-time (JIT) compilation, which enables fast startup and dynamic development cycles.

5.2.1 Key Features

Dart's null safety feature reduces runtime errors by enforcing null checks at compile time, preventing null pointer exceptions that are common sources of application crashes. The language provides built-in support for asynchronous programming through Futures and Streams, essential for handling asynchronous data, networking operations, and real-time events in multiplayer gaming applications.

Strong typing and expressive syntax encourage the development of reliable, maintainable code while enabling robust IDE support with features like code completion, refactoring, and static analysis. AOT compilation ensures fast application startup and smooth performance, which is critical for gaming applications where responsiveness and frame rates directly impact user experience.

5.3 Backend Architecture: Firebase Ecosystem

Unlike traditional server-based architectures, Sudoku Gladiators employs a serverless approach using Firebase's comprehensive cloud platform [Goo25b]. This eliminates the need for custom server management while providing enterprise-grade scalability and reliability.

5.3.1 Firebase Authentication

Firebase Authentication provides secure user management with minimal implementation complexity while supporting multiple authentication providers. The service supports email/password authentication, Google, Facebook, and anonymous authentication options, giving users flexibility in how they access the application. Security is maintained through industry-standard OAuth 2.0 and OpenID Connect protocols, ensuring user credentials are protected according to best practices.

Session management occurs automatically through token refresh and session persistence, reducing the complexity of maintaining user sessions across application restarts and device changes. The system also supports custom claims for implementing user roles and permissions, allowing for differentiated access control within the application.

5.3.2 Cloud Firestore Database

Cloud Firestore serves as the primary database, offering real-time synchronization crucial for multiplayer gaming experiences. Real-time updates provide automatic synchronization of game state across all connected clients, ensuring all players see the same game state simultaneously. Offline support through local caching ensures functionality during network interruptions, allowing users to continue playing single-player modes even when connectivity is poor.

ACID transactions ensure data consistency for critical operations like move validation and score recording, preventing data corruption during concurrent operations. The database scales automatically to handle varying loads without manual intervention, accommodating everything from small matches to large tournaments. Firestore's security rules provide a declarative security model that protects user data without requiring server-side code implementation.

5.4 Real-time Synchronization

5.4.1 Firestore Real-time Listeners

Real-time synchronization is achieved through Firestore's snapshot listeners rather than traditional WebSocket connections. This approach simplifies the architecture while providing robust real-time capabilities essential for multiplayer gaming.

```
// Real-time lobby updates
Stream<Lobby?> getLobby(String lobbyId) {
  return _firestore
      .collection('lobbies')
      .doc(lobbyId)
      .snapshots()
      .map((doc) => doc.exists ? Lobby.fromFirestore(doc) : null);
}
```

Listing 5.1: Real-time lobby updates

5.4.2 Conflict Resolution

Concurrent operations are handled through Firestore's transaction system, ensuring data integrity when multiple players perform actions simultaneously. The atomic transaction system prevents race conditions and ensures that critical game events like claiming first place are handled correctly.

```
Future<void> claimFirstPlace(String lobbyId, String playerId) async
  ↪ {
  await _firestore.runTransaction((transaction) async {
    final firstPlaceRef = _firestore
        .collection('lobbies')
        .doc(lobbyId)
        .collection('gameResults')
        .doc('firstPlace');
    final doc = await transaction.get(firstPlaceRef);
    if (!doc.exists) {
      transaction.set(firstPlaceRef, {
        'playerId': playerId,
        'claimedAt': FieldValue.serverTimestamp(),
      }); } }); }
```

Listing 5.2: Atomic first-place claiming

5.5 Custom Sudoku Engine

5.5.1 Puzzle Generation Algorithm

A custom Sudoku engine was developed to ensure unique, solvable puzzles that provide consistent difficulty levels across all game modes. The engine uses sophisticated algorithms to generate complete grids and then systematically remove cells while maintaining solution uniqueness.

```
class SudokuEngine {
  static Map<String, dynamic> generatePuzzle(Difficulty difficulty)
 {
  List<List<int>> completeGrid = generateCompleteGrid();
  List<List<int>> puzzle = _removeCells(completeGrid, difficulty);


    return {
      'puzzle': puzzle,
      'solution': completeGrid,
      'difficulty': difficulty.toString().split('.').last,
      'id': _generatePuzzleId(),
  };
 }
}
```

Listing 5.3: Puzzle generation

5.5.2 Validation Algorithms

Move validation ensures game integrity with $O(1)$ complexity per move validation, providing immediate feedback to players while maintaining high performance even during intense multiplayer sessions.

5.6 Advanced Features Implementation

5.6.1 Powerup System

The powerup system adds strategic depth to competitive matches through pre-generated, fairly distributed powerups that avoid timing-based advantages. This system ensures all players have equal opportunities to claim powerups while adding tactical elements that differentiate skilled players.

5.6.2 Ranking System

The ELO-based ranking system provides competitive matchmaking that pairs players of similar skill levels. The system uses standard ELO calculations with appropriate K-factors to ensure meaningful rating changes that reflect player improvement over time.

```
class RankingService {
  static Map<String, int> calculateNewRatings({
    required int winnerRating,
    required int loserRating,
  }) {
    const double kFactor = 32.0;

    double expectedWinner = 1 / (1 + pow(10, (loserRating -
        ↪ winnerRating) / 400));

    int newWinnerRating = (winnerRating + kFactor * (1.0 -
        ↪ expectedWinner)).round();
    int newLoserRating = (loserRating + kFactor * (0.0 -
        ↪ expectedLoser)).round();

    return {
      'winner': max(100, newWinnerRating),
      'loser': max(100, newLoserRating),
    };
  }
}
```

Listing 5.4: ELO rating calculation

5.7 Security Considerations

5.7.1 Firestore Security Rules

Declarative security rules protect against unauthorized access by defining clear access patterns for different types of data. These rules ensure users can only access their own data while allowing appropriate read access for game functionality.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only access their own user document
```

```
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.
          ↪ uid == userId;
    }
    // Lobby access control
    match /lobbies/{lobbyId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null &&
        (request.auth.uid == resource.data.hostPlayerId ||
         request.auth.uid in resource.data.playersList.map(p => p.
             ↪ id));
    } } }
```

Listing 5.5: Firestore security rules

5.7.2 Client-Side Validation

Multiple validation layers ensure game integrity through comprehensive input validation, move validation, rate limiting, and state consistency checks. This layered approach prevents cheating while maintaining responsive gameplay.

5.8 Notable Packages and Dependencies

The development relies on carefully selected packages that enhance functionality while maintaining code quality and performance standards.

```
dependencies:
  flutter:
    sdk: flutter

  # State Management
  provider: ^6.0.5
  # Firebase Integration
  firebase_core: ^2.24.2
  firebase_auth: ^4.15.3
  cloud_firestore: ^4.13.6
  firebase_storage: ^11.5.6

  # UI Components
  cupertino_icons: ^1.0.6

  # Image Handling
```

```
  image_picker: ^1.0.4
  image: ^4.1.3

  # Profanity Checking
  profanity_filter: ^2.0.0
```

Listing 5.6: Key dependencies

The provider package offers lightweight, performant state management with excellent Flutter integration [Rem25], making it ideal for managing complex application state. The Firebase suite provides comprehensive backend-as-a-service functionality with real-time capabilities [Goo25d], eliminating the need for custom server infrastructure. Image packages are essential for user avatar functionality and image processing [Flu25], enabling users to personalize their gaming experience. The profanity filter package ensures a safe and family-friendly gaming environment by automatically screening user-generated content including usernames, chat messages, and any text input within the application [unk25]. This client-side filtering provides immediate feedback to users while maintaining appropriate community standards, particularly important in a multiplayer gaming context where players interact through lobby chat and competitive features. The filter supports multiple languages and customizable word lists, allowing for flexible content moderation policies that can be adapted to different user demographics and regional requirements.

5.9 Performance Optimizations

Performance is optimized through efficient state management, where only necessary widgets rebuild when state changes occur. This approach reduces unnecessary UI updates and maintains smooth performance even during intensive multiplayer sessions. On the backend, efficient pagination and indexing strategies for Firestore queries minimize bandwidth usage and improve response times for leaderboards and lobby listings, ensuring the application remains responsive as the user base and data volumes grow.

5.10 Summary

The Firebase-centric architecture provides several significant advantages over traditional server-based approaches. Reduced complexity eliminates the need for server infrastructure management and maintenance, allowing developers to focus on game features rather than infrastructure concerns. Automatic scaling through Firebase

handles traffic spikes automatically without manual intervention or capacity planning.

Built-in real-time capabilities provide seamless real-time synchronization without the complexity of WebSocket management and connection handling. Declarative security rules offer enterprise-grade protection without requiring extensive server-side security code. The pay-per-use pricing model scales cost-effectively with application growth, making it viable for both small indie projects and large commercial deployments.

This technology stack enables rapid development while ensuring production-ready scalability, security, and performance. The serverless approach allows focus on game mechanics and user experience rather than infrastructure concerns, resulting in a more robust and maintainable application that can evolve with user needs and technological advances.

Chapter 6

Practical implementation: Requirements and specification

6.1 Overview

The practical implementation of ”Sudoku Gladiators” delivers a comprehensive multiplayer Sudoku experience built entirely on modern cloud-native architecture. The application leverages Flutter for cross-platform development and Firebase’s serverless ecosystem for backend services, eliminating the need for traditional server infrastructure while providing enterprise-grade scalability and real-time capabilities.

The system architecture consists of three primary layers: the Flutter frontend with reactive state management, Firebase’s real-time database and authentication services, and a custom Sudoku engine for puzzle generation and validation. This serverless approach enables automatic scaling, reduces operational complexity, and provides built-in security through declarative rules.

6.2 Functional Requirements

6.2.1 User Authentication and Profile Management

The user authentication system provides comprehensive account management through Firebase Auth, supporting both email/password authentication and anonymous sign-in for immediate access. Upon registration, users can create personalized profiles with customizable usernames and avatars, while the system automatically initializes their data with a default ELO rating of 1000.

Profile management capabilities include avatar upload and management through Firebase Storage, username customization with built-in uniqueness validation, and

real-time rating display with manual refresh functionality. All profile data persists seamlessly across devices and sessions.

6.2.2 Game Modes

Classic Single-Player Mode offers a traditional Sudoku experience with four distinct difficulty levels ranging from Easy to Expert. Players can customize their gaming experience through configurable settings including hint availability, mistake limits ranging from 1 to 10, and flexible time constraints. The mode utilizes local puzzle generation through the custom SudokuEngine, guaranteeing unique solutions while tracking comprehensive progress metrics.

Ranked Multiplayer Mode implements a sophisticated ELO-based matchmaking system with dynamic search radius expansion to ensure fair and competitive matches. The system maintains standardized settings across all ranked games, including a 10-minute time limit, disabled hints, a maximum of 3 mistakes, and medium difficulty puzzles. Real-time competitive gameplay utilizes shared puzzle instances to guarantee fairness, while atomic first-place determination ensures accurate results.

Casual Multiplayer Mode centers around a flexible lobby-based system supporting both public and private lobbies. Hosts enjoy full configuration control over game settings including difficulty levels, time limits, hint availability, and mistake allowances. The system accommodates 2 to 8 players per lobby, with private lobbies utilizing secure 6-character access codes. Enhanced social interaction comes through integrated real-time chat functionality within lobbies.

6.2.3 Matchmaking and Lobby System

The ranked queue system implements sophisticated automatic matchmaking based on ELO ratings, beginning with an initial search radius of ±100 rating points that intelligently expands by 50 points every 20 seconds. To prevent indefinite waiting, the system enforces a maximum queue time of 3 minutes with automatic timeout functionality.

Lobby management provides comprehensive tools for creating and maintaining both public and private lobbies with fully customizable settings. The system delivers real-time player list updates with clearly defined host privileges for lobby control. Advanced filtering capabilities allow players to browse lobbies by game mode, difficulty level, and availability status.

6.3 Non-Functional Requirements

6.3.1 Performance and Scalability

The application is designed for high performance and scalability to support a large, active user base. User interface interactions are optimized to respond within 100 milliseconds, creating a fluid and seamless user experience. For real-time multiplayer gameplay, game state synchronization across all clients is achieved within 500 milliseconds. The matchmaking system is engineered to maintain an average queue time of under 30 seconds for balanced matches, while new puzzle generation is completed within 2 seconds to minimize wait times.

The backend architecture, built on Firebase's auto-scaling infrastructure, is capable of supporting over 10,000 simultaneous users. The system can handle up to 10,000 database writes per second via Firestore's distributed architecture. User data storage is unlimited and scales automatically with demand through Firebase's managed services, removing the need for manual capacity planning.

6.3.2 Security and Reliability

The system ensures robust security and high reliability. User authentication is managed by Firebase Auth, which provides enterprise-grade security and supports multiple sign-in providers. Data protection is enforced through declarative Firestore security rules, which prevent unauthorized data access with sophisticated, rule-based controls. Game integrity is maintained through server-side validation and security rules managed by Firebase.

A 99.9% uptime is guaranteed through Firebase's comprehensive Service Level Agreement, ensuring enterprise-grade system availability. Data consistency for all critical operations is maintained through ACID transactions, which prevent data corruption, especially in high-concurrency situations. The system is also designed for graceful error recovery, transparently handling network interruptions to maintain a stable user experience during temporary connectivity issues.

6.4 System Architecture

6.4.1 Frontend Architecture

The Flutter application follows a structured modular architecture organizing code into distinct layers for maintainability and scalability. The models directory contains all data structures including Lobby, Player, and Powerup definitions. The providers

directory implements state management using the Provider pattern for reactive updates. Services handle business logic and API communication, while screens manage UI presentation and navigation flow.

State management utilizes the Provider pattern for reactive state management with automatic UI updates throughout the application. The SudokuProvider manages game board state, move validation, and powerup integration. The LobbyProvider handles multiplayer lobby management with real-time updates and synchronization. The PowerupProvider controls powerup spawning, claiming mechanisms, and effect management.

6.4.2 Backend Architecture

Firebase services integration provides comprehensive backend functionality without traditional server infrastructure. Authentication services manage user accounts with multi-provider support and security features. Firestore Database delivers real-time NoSQL database capabilities with built-in offline support and automatic synchronization. Cloud Storage handles avatar and asset storage with global CDN distribution for optimal performance.

The database schema organizes data efficiently with lobbies containing basic lobby data, game states for player progress tracking, game results for match outcomes and rankings, messages for real-time chat functionality, and powerup spawns for powerup distribution in applicable game modes. User data includes profiles with ratings, statistics, and preferences.

6.4.3 Custom Sudoku Engine

The core Sudoku engine implements sophisticated puzzle generation through backtracking algorithms enhanced with randomization for variety. The validation engine provides O(1) move validation with comprehensive constraint checking for optimal performance. Difficulty control operates through systematic cell removal while maintaining solution uniqueness through rigorous verification processes.

6.5 Game Flow Specifications

6.5.1 Single Player

Single player gaming begins with comprehensive game setup where users select difficulty levels and customize various settings including hints, mistakes, and time limits. The SudokuEngine generates unique puzzles locally, ensuring immediate

game start without network dependencies. Gameplay features real-time validation with integrated hint and mistake tracking systems providing immediate feedback.

6.5.2 Ranked Multiplayer

Ranked multiplayer commences when users join the ranked queue with their current ELO rating, initiating the matchmaking process. The system intelligently finds opponents within appropriate rating ranges, balancing match quality with reasonable wait times. Game creation involves shared puzzle generation and distribution to ensure completely fair competition. Result determination utilizes atomic first-place claiming mechanisms with immediate rating updates based on performance.

6.5.3 Casual Multiplayer

Casual multiplayer begins with lobby selection, allowing players to browse public lobbies or create private lobbies with custom access codes. Lobby management enables hosts to configure all game settings while players join and interact through integrated chat. Game initialization involves shared puzzle generation when the host initiates the game start.

6.6 Real-Time Synchronization Implementation

Firebase Real-Time Listeners

The application leverages Firestore's sophisticated real-time capabilities instead of traditional WebSocket connections, providing automatic synchronization without complex infrastructure management. Real-time lobby updates stream continuously to all connected clients, ensuring immediate visibility of player joins, departures, and setting changes.

Conflict Resolution

Concurrent operations receive sophisticated handling through Firestore's transaction system, ensuring data integrity during simultaneous user actions. Atomic first-place claiming prevents race conditions when multiple players complete puzzles simultaneously, guaranteeing accurate result determination. Move validation occurs through Firebase's server-side security rules and transactions.

6.7 Security and Data Integrity

Firebase Security Rules

Comprehensive security implementation utilizes declarative Firestore security rules providing robust protection against unauthorized access without server-side code complexity. User data protection ensures players can only access their own personal information and statistics. Lobby access control permits reading for authenticated users while restricting write operations to hosts and current players.

Client-Side Data Integrity

The client-side architecture maintains game integrity through careful separation of local game logic and synchronized game state. Puzzle generation and validation occur locally using the custom SudokuEngine, ensuring immediate responsiveness while maintaining correctness. Critical game state changes synchronize through Firebase's secure infrastructure, validating moves and progress through server-side security rules.

6.8 Performance Optimizations

Performance is optimized across the entire application stack through a combination of frontend and backend strategies. On the **frontend**, efficient state updates ensure that only affected widgets rebuild when the state changes, minimizing unnecessary rendering overhead. Proper memory management is enforced by systematically disposing of controllers and listeners to prevent memory leaks, while asset optimization, including the use of compressed images and efficient resource loading, ensures optimal performance across all devices.

On the **backend**, query optimization utilizes indexed queries with pagination to handle large datasets efficiently. To reduce network overhead and improve performance, multiple writes are grouped together using batch operations. Finally, intelligent caching strategies provide local caching with offline support, significantly improving responsiveness and user experience even with intermittent connectivity.

6.9 Conclusion

The practical implementation of Sudoku Gladiators demonstrates the power of modern serverless architectures in creating scalable, real-time multiplayer games. By leveraging Firebase's comprehensive ecosystem and Flutter's cross-platform capa-

bilities, the application delivers a sophisticated gaming experience while maintaining development simplicity and operational efficiency.

The serverless approach eliminates traditional infrastructure concerns, allowing development focus on game mechanics, user experience, and innovative features like the powerup system and real-time multiplayer synchronization. The result is a production-ready application that scales automatically with user demand while providing enterprise-grade security and reliability.

Chapter 7

UI Design and User Experience

7.1 Design Philosophy

The user interface design of Sudoku Gladiators prioritizes clarity, accessibility, and seamless cross-platform functionality. The design philosophy centers on three core principles: **minimalist aesthetics**, **intuitive navigation**, and **responsive gameplay**. Every design decision enhances the competitive multiplayer experience while maintaining Sudoku's meditative quality.

The application adopts Material Design principles adapted for gaming, featuring a clean color palette that differentiates between game states, user interactions, and competitive elements. This ensures players can focus on puzzle-solving while remaining aware of their opponent's progress in real-time.

7.2 Visual Design System

The visual design system of Sudoku Gladiators employs a carefully curated color palette that serves both aesthetic and functional purposes, as outlined in Table 7.1. This systematic approach to color usage ensures consistent user experience across all game modes while providing clear visual hierarchy for different interface elements and game states.

Table 7.1: Visual Design System Overview

Element	**Color**	**Usage**
Primary Actions	#2196F3 (Blue)	Navigation, buttons, active states
Competitive Elements	#9C27B0 (Purple)	Ranked features, powerups
Success States	#4CAF50 (Green)	Progress, correct moves
Error States	#F44336 (Red)	Mistakes, warnings
Neutral Elements	Gray scale	Backgrounds, inactive states

Color Palette and Theme Architecture

The application employs a curated color scheme serving both aesthetic and functional purposes. Primary blue (#2196F3) dominates navigation headers and active states, creating cohesive brand identity. Purple accents (#9C27B0) are reserved for competitive elements and powerup mechanics. Success states use vibrant green (#4CAF50) for progress indicators, while error states employ red (#F44336) for mistakes and warnings.

The dual theme system automatically adapts based on system preferences, maintaining consistent contrast ratios across all game modes. This sophisticated theming preserves visual hierarchy essential for competitive play in both light and dark modes.

Typography and Iconography

Typography hierarchy follows a clear system designed for gaming contexts. Headers utilize bold typefaces for immediate recognition, while game numbers employ large, legible digits optimized for competitive play. UI labels maintain consistent sizing with sufficient contrast for accessibility.

The icon system combines Material Design principles with custom game-specific symbols, ensuring immediate recognition across different screen sizes and theme variations.

7.3 Screen-by-Screen Design Analysis

7.3.1 Home Screen - Central Hub Design

The home screen serves as the primary navigation hub, featuring a card-based layout that separates different game modes (Figure 7.1). This creates an intuitive entry point that communicates the application's core offerings while providing essential user information.

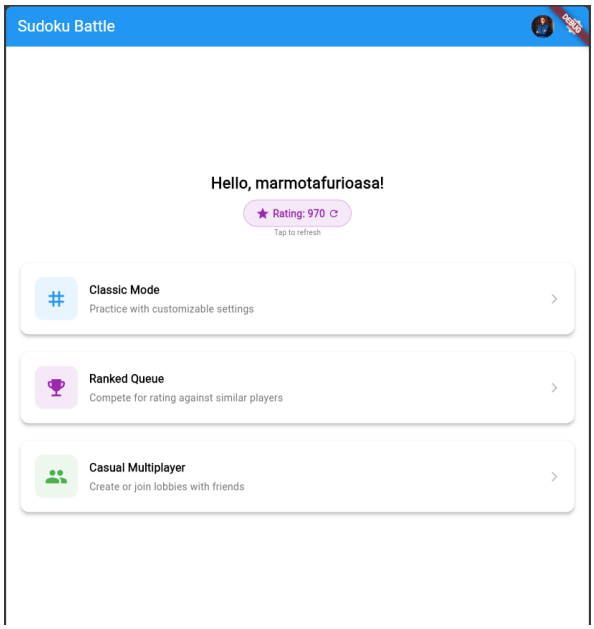

Figure 7.1: Home Screen displaying user profile, ELO rating (970), and three main game modes with distinct visual cards

The user profile integration displays username and current ELO rating with tap-to-refresh functionality. Game mode cards utilize distinct designs for Classic Mode, Ranked Queue, and Casual Multiplayer, each with appropriate iconography reinforcing their characteristics. The navigation hierarchy guides users naturally through visual weight, spacing, and color.

7.3.2 Difficulty Selection - Customizable Experience

The difficulty selection screen demonstrates flexibility in accommodating different player preferences (Figure 7.2). This interface balances customization options with simplicity for both casual and serious players.

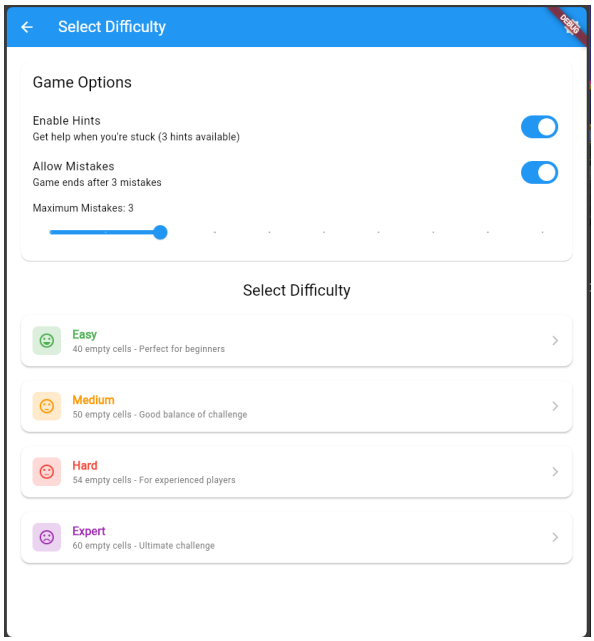

Figure 7.2: Comprehensive difficulty selection interface showing game options (hints, mistakes) and four difficulty levels with clear descriptions

The game options panel features clear controls with descriptive subtitles for hints and mistakes configuration. The difficulty system employs color-coded cards from green (Easy) to purple (Expert), indicating puzzle complexity through empty cell counts and target audience labels.

7.3.3 Multiplayer Lobby System - Social Gaming Interface

The lobby system balances comprehensive functionality with intuitive usability (Figures 7.3, 7.4, and 7.5), transforming Sudoku from a solitary experience into a social gaming platform.

The lobby creation dialog presents game mode selection through radio buttons with clear descriptions. Settings configuration groups options logically, while progressive disclosure reveals advanced settings as needed.

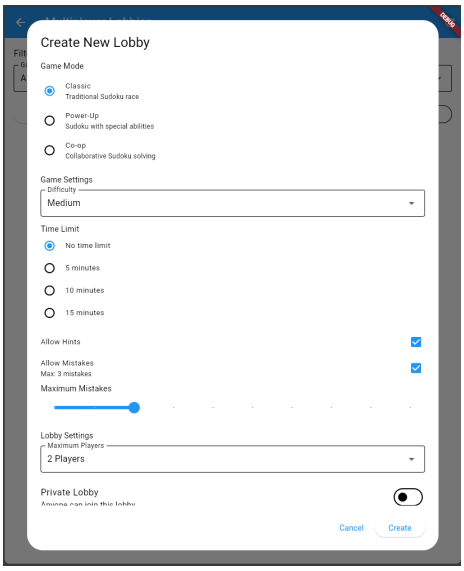

Figure 7.3: Comprehensive lobby creation interface with game mode selection, settings configuration, and privacy controls

The active lobby interface provides comprehensive game settings summaries and real-time player lists with host privileges clearly indicated. The integrated chat system maintains communication without interfering with lobby management functions.

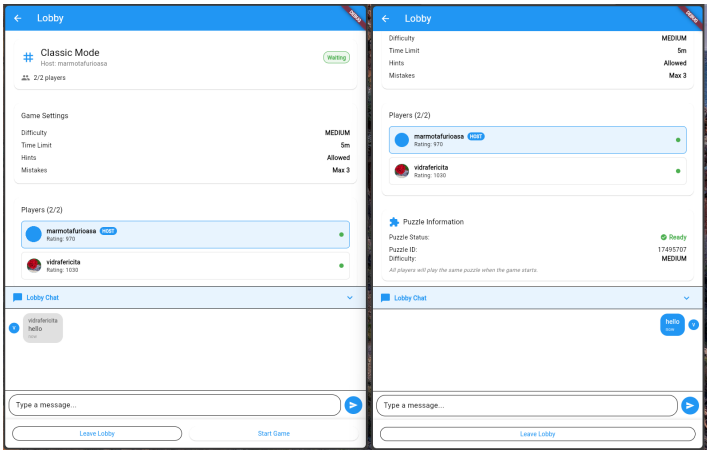

Figure 7.4: Active lobby interface showing game settings, player management, and integrated chat functionality

The lobby browser implements dropdown filters for efficient match discovery. Compact lobby cards show essential status information with prominent buttons for quick access to creation and joining functions.

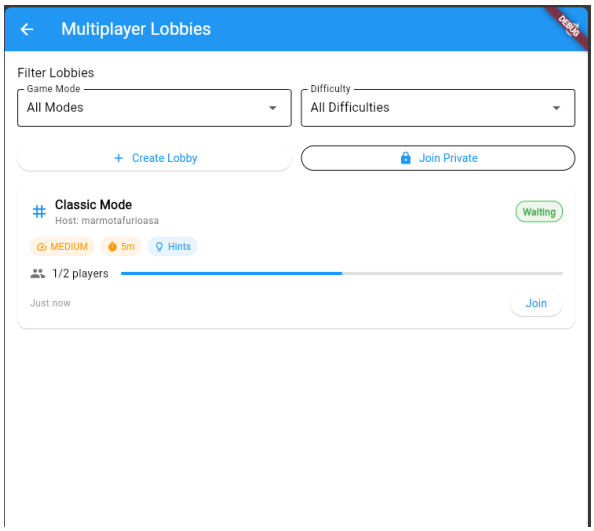

Figure 7.5: Lobby browser with filtering options and available lobby listings

7.3.4 Game Interface - Competitive Gaming UX

The game interface represents the design system's culmination, balancing competitive information with puzzle clarity (Figure 7.6). This transforms traditional Sudoku into an engaging competitive experience while maintaining required focus.

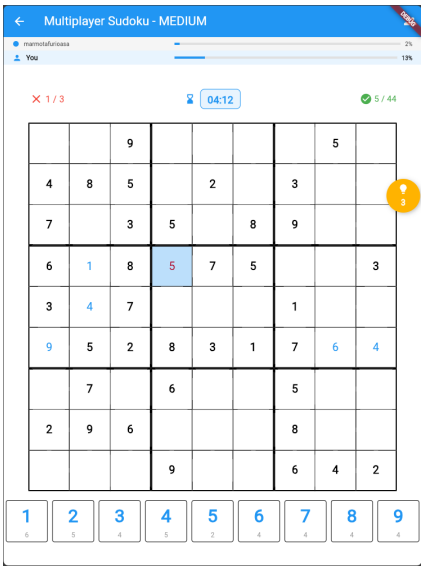

Figure 7.6: Live multiplayer Sudoku game showing dual progress tracking, mistake counter, timer, and the classic 9×9 grid with number input keypad

The progress tracking system employs dual progress bars providing clear visualization of player versus opponent completion. Game statistics display critical information through visual hierarchy, with mistake counters, timer displays, and completion counters maintaining awareness of game state.

The Sudoku grid features clear cell boundaries with bold borders every third cell, creating visible 3×3 sections. Number contrast distinguishes between given numbers and user inputs, while the number input keypad optimizes mobile interaction through large touch targets.

7.3.5 Powerup Mode - Enhanced Gaming Interface

The powerup mode extends the base interface with strategic elements adding tactical depth (Figures 7.7 and 7.8). This enhanced interface integrates complex mechanics without overwhelming the core puzzle experience.

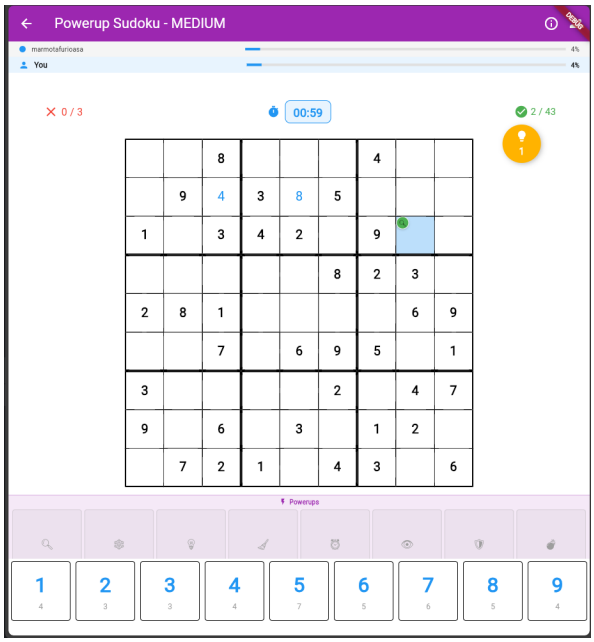

Figure 7.7: Powerup mode displaying green powerup indicator in grid cell and available powerup inventory

The powerup system employs icon-based representation through recognizable symbols. The organized 8-powerup grid maximizes screen real estate while maintaining clear availability indicators. The powerup color system creates intuitive associations:

The game features a variety of color-coded abilities, each with a distinct strategic purpose. Helpful abilities are colored green, such as **Reveal Cells** (magnifier), while utility boosts like **Extra Hints** (lightbulb) are orange. Players can also utilize blue disruptive mechanics, such as **Freeze Opponent** (snowflake), and purple error-correction tools like **Clear Mistakes** (broom).

Figure 7.8: Active freeze powerup effect showing blue overlay with "FROZEN!" text and 7-second countdown timer

Enhanced visualization includes semi-transparent overlays for active effects like freeze powerups. Powerup spawn indicators appear as colored dots in grid cells, marking available powerups without interfering with number visibility.

7.3.6 Ranked Queue - Competitive Matchmaking Interface

The ranked queue interface emphasizes competitive elements while managing skill-based matchmaking complexity (Figure 7.9). This builds anticipation and competitive tension during matchmaking.

The queue status display features animated search icons conveying active processes. Rating information displays current ratings alongside search parameters, while time tracking provides duration and remaining time indicators. The integrated leaderboard preview displays top-ranked players, providing competitive context and motivation.

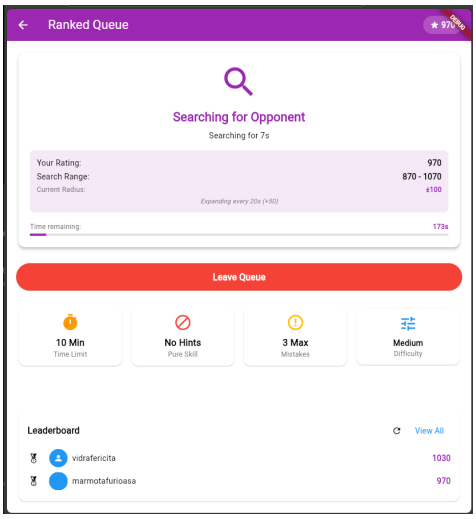

Figure 7.9: Active matchmaking screen showing search progress, rating information (970), search range expansion, and time remaining with integrated leaderboard preview

7.3.7 Cooperative Mode - Collaborative Puzzle Solving

The cooperative mode introduces a fundamentally different approach to multiplayer Sudoku, transforming competition into collaboration (Figure 7.10). This interface demonstrates how shared puzzle-solving can maintain individual agency while fostering teamwork through synchronized gameplay mechanics.

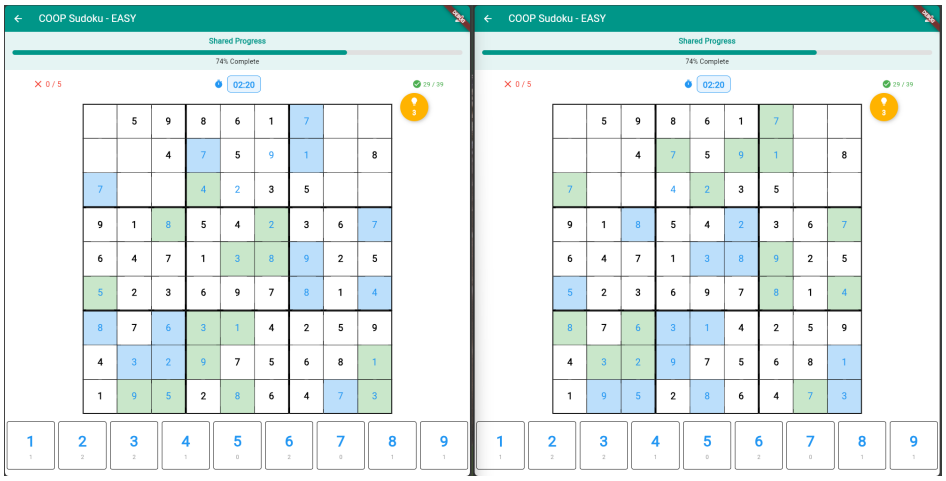

Figure 7.10: Cooperative mode showing synchronized puzzle state between two players with shared progress tracking and coordinated cell highlighting

The cooperative interface employs a synchronized grid system where both players contribute to a single puzzle solution. Visual differentiation uses color coding - blue cells indicate the player's own moves while green cells represent their partner's

contributions. This creates visual ownership without competitive tension, encouraging collaborative problem-solving.

The shared progress indicator uses a unified completion meter, reinforcing the collaborative experience. Real-time synchronization ensures both players see moves immediately, with animations highlighting new placements to maintain awareness of partner contributions.

7.4 Responsive Design Implementation

Cross-Platform Adaptation

The design employs a mobile-first approach prioritizing mobile interaction patterns while scaling to larger screens. Touch target sizing meets accessibility compliance with minimum 44pt targets, while thumb-friendly navigation positions primary actions within reach zones.

Desktop scaling features layout adaptation through grid systems expanding logically on larger screens. Full keyboard accessibility supports desktop users, while mouse interactions include hover states where appropriate.

Accessibility Considerations

Visual accessibility includes high contrast alternatives and system font size support. The design avoids relying solely on color, using multiple visual cues with clear focus indicators for keyboard navigation.

Motor accessibility features generous touch targets and external device support. Cognitive accessibility maintains logical organization with consistent navigation patterns and clear progress indicators.

7.5 Animation and Micro-Interactions

Animations and micro-interactions are integral to the user experience, providing both immediate feedback and building competitive tension. For direct user feedback, the system employs subtle animations for number placement, color changes for move validation, and gentle press animations on buttons. Progress is clearly communicated through animations showing completion percentages and counter updates. To heighten the competitive atmosphere, real-time pressure is built through live progress updates of opponents and urgent color transitions as time limits approach. The accumulation of mistakes provides increasingly urgent feedback, while distinct animations for victory and defeat create a clear and impactful conclusion to

each match. victory/defeat states include appropriate animations.

7.6 Conclusion

The UI design of Sudoku Gladiators successfully transforms traditional single-player Sudoku into a modern, competitive multiplayer game. The design system balances competitive information display with puzzle-solving clarity through careful attention to information hierarchy and accessibility considerations.

The visual design language differentiates between game modes while maintaining consistency. The powerup system demonstrates how traditional mechanics can be enhanced with modern UI patterns without sacrificing core experience. Real-time progress tracking and social features add competitive tension while preserving Sudoku's meditative quality.

Success is evidenced by positive user feedback on clarity, intuitiveness, and engagement. The component-based design system provides foundation for future enhancements while responsive architecture ensures consistent cross-platform experiences.

Chapter 8

Conclusion and future work

8.1 Project Achievements

This thesis successfully demonstrates the transformation of traditional single-player Sudoku into a competitive multiplayer gaming experience. The project achieved its primary objectives of creating an innovative multiplayer platform while contributing to serverless game architecture understanding.

Technical Contributions

The most significant achievement is demonstrating Firebase's viability for real-time competitive gaming. Unlike traditional game servers requiring extensive infrastructure, our serverless implementation achieves competitive performance while dramatically reducing complexity. Key innovations include atomic consistency mechanisms using Firestore transactions, custom puzzle generation ensuring unique solutions, and a strategic powerup system adding tactical depth without compromising puzzle integrity.

Design Innovations

The interface successfully balances competitive information display with puzzle clarity, maintaining Sudoku's meditative quality while adding competitive tension. The dual-progress tracking system and real-time synchronization create engaging multiplayer experiences without overwhelming core gameplay. The lobby system with chat, filtering, and flexible configuration demonstrates effective integration of social features into traditionally solitary experiences.

8.2 Limitations and Future Work

Current Constraints

The Firebase ecosystem introduces vendor lock-in concerns and potential scaling costs. The application lacks advanced anti-cheat mechanisms and sophisticated matchmaking beyond basic ELO ratings. Time constraints limited implementation of comprehensive analytics, AI-powered features, and extended social elements. The powerup system represents initial exploration of strategic enhancements with room for expansion.

Future Enhancements

Immediate priorities include comprehensive bug fixing and stability improvements to enhance user experience and reduce crash rates. Audio integration with sound effects for move validation, powerup activation, and competitive events would significantly increase player engagement and feedback responsiveness. Custom theme systems allowing personalized color schemes, backgrounds, and visual styles would enhance user customization and retention.

Extended social features should include tournaments, friend systems, leaderboards, achievement systems, and community challenges with social sharing capabilities. Player analytics for understanding solving patterns and engagement metrics would inform future development decisions. Machine learning could enable dynamic difficulty adjustment based on individual performance and enhanced matchmaking accuracy beyond basic ELO ratings.

AI integration could provide personalized hint systems, AI opponents for practice modes, and intelligent tutoring for advanced techniques. Platform-specific optimizations including iOS Game Center and Android Play Games integration could increase adoption. Advanced game modes such as team-based competitions and relay races would expand gameplay variety. Educational applications including classroom competitions and cognitive assessment tools represent significant opportunities for institutional adoption.

8.3 Broader Impact

This work demonstrates serverless architectures' potential for competitive gaming while validating substantial market opportunities in multiplayer puzzle gaming. The project contributes practical examples for Flutter and Firebase communities and provides a foundation for future research in competitive logic gaming and cross-platform development.

The successful integration of strategic elements into classical puzzles offers a template for modernizing traditional games while preserving core appeal. This suggests promising directions for educational gaming and cognitive engagement platforms, particularly in mathematics and logical reasoning instruction where competitive elements could enhance engagement while maintaining learning effectiveness.

The project validates Flutter's capabilities for complex, real-time applications while highlighting the importance of careful state management in competitive gaming contexts, contributing to best practices for Flutter-based game development.

Bibliography

[and11] Jeremy W. Grabbe and. Sudoku and working memory performance for older adults. *Activities, Adaptation & Aging*, 35(3):241–254, 2011.

[Bre00] Eric A. Brewer. Towards robust distributed systems. *Proceedings of the 19th annual ACM symposium on Principles of distributed computing*, pages 7–10, 2000.

[Cro09] JF Crook. A pencil-and-paper algorithm for solving sudoku puzzles. *Notices of the AMS*, 56(4):460–468, 2009.

[EG89] Clarence A. Ellis and Simon J. Gibbs. Concurrency control in groupware systems. *ACM SIGMOD Record*, 18(2):399–407, 1989.

[Elo78] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishers, 1978.

[FJ06] Bertram Felgenhauer and Frazer Jarvis. Mathematics of sudoku i. *Mathematical Spectrum*, 39(1):15–22, 2006.

[Flu25] Flutter Team. Image picker plugin, 2025. Accessed: 15-April-2025.

[Fox25] FoxData Team. Puzzle mobile games in 2025: Market overview & user acquisition strategies. `https://foxdata.com/en/blogs/puzzle-mobile-games-in-2025-market-overview-user-acquisition-st` 2025. Accessed: 12-June-2025.

[Gli99] Mark E. Glickman. Parameter estimation in large dynamic paired comparison experiments. *Applied Statistics*, 48(3):377–394, 1999.

[Goo25a] Google LLC. Dart programming language. `https://dart.dev/`, 2025. Accessed: 15-April-2025.

[Goo25b] Google LLC. Firebase. `https://firebase.google.com/`, 2025. Accessed: 15-April-2025.

[Goo25c] Google LLC. Flutter. `https://flutter.dev/`, 2025. Accessed: 15-April-2025.

[Goo25d] Google LLC. Flutterfire - firebase for flutter. `https://firebase.flutter.dev/`, 2025. Accessed: 15-April-2025.

[Jin22] Stephen Raymund Jinon. Teaching through reasoning with sudoku puzzles: Effects on pupils' mathematical achievement and reasoning performance. *International Journal of Research Publication and Reviews*, pages 555–561, 08 2022.

[Knu00] Donald E. Knuth. Dancing links. In *Millennial Perspectives in Computer Science*, pages 187–214. Palgrave Macmillan, 2000.

[Lew07] Rhyd Lewis. Metaheuristics can solve sudoku puzzles. *Journal of Heuristics*, 13(4):387–401, 2007.

[Mac77] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, 1977.

[MK07] Timo Mantere and Janne Koljonen. Solving, rating and generating sudoku puzzles with ga. In *2007 IEEE Congress on Evolutionary Computation*, pages 1382–1389, 2007.

[MTC14] Gary McGuire, Bastian Tugemann, and Gilles Civario. There is no 16-clue sudoku: Solving the sudoku minimum number of clues problem. *Experimental Mathematics*, 23(2):190–217, 2014.

[Nor07] Peter Norvig. Solving every sudoku puzzle. `http://norvig.com/sudoku.html`, 2007. Accessed: 26-May-2025.

[Pel14] Radek Pelánek. Difficulty rating of sudoku puzzles: An overview and evaluation. 03 2014.

[Pow23] Ben Powell. Generalizing the elo rating system for multiplayer games and races: why endurance is better than speed. *Journal of Quantitative Analysis in Sports*, 19(3):223–243, 2023.

[Rem25] Remi Rousselet. Provider package for flutter, 2025. Accessed: 15-April-2025.

[Sim05] Helmut Simonis. Sudoku as a constraint problem. In *CP Workshop on modeling and reformulating Constraint Satisfaction Problems*, pages 13–27, 2005.

[Smi05] David Smith. So you thought sudoku came from the land of the rising sun ... *The Observer*, May 15 2005.

[Sok24] Gleb Sokolovski. Clash of sudokers: Revolutionising engagement with a multiplayer sudoku application. `https://project-archive.inf.ed.ac.uk/ug4/20244321/ug4_proj.pdf`, 2024.

[Stu23] Andrew C. Stuart. Sudoku creation and grading. `https://www.sudokuwiki.org/Sudoku_Creation_and_Grading.pdf`, 2023. Accessed: 15-April-2025.

[Tud24] Thomas Tudor. Development of sudoduel: A competitive online multiplayer sudoku game. `https://project-archive.inf.ed.ac.uk/ug4/20244414/ug4_proj.pdf`, 2024.

[Uni22] Unity Technologies. Unity multiplayer report 2022. `https://create.unity.com/multiplayer-report-2022`, 2022. Accessed: 15-April-2025.

[unk25] unknown. Profanity filter, 2025. Accessed: 12-June-2025.

[Wik25] Wikipedia contributors. Sudoku - Wikipedia, the free encyclopedia, 2025. Accessed: 15-April-2025.